ASSIGNMENT 02

Rubik's Cube Race

Model thinking & reasoning quality vs. latency | OpenRouter APIs

The central question

Is a model that makes decisions in milliseconds more useful than a frontier LLM that spends longer planning, if the slower model can solve the task with fewer moves? Participants will test this trade-off using a repeatable Rubik's Cube benchmark.

This assignment converts an informal observation into a controlled experiment. The aim is to measure the behavior of the different style of models, not to assume that any model will win in advance.

1. The challenge: Can Fast Decision Models Beat Frontier LLMs at Solving a Rubik's Cube?

Build an AI-powered Rubik's Cube game and benchmark four models through OpenRouter. All models must begin from the same randomly scrambled cube state. Each model chooses moves until it solves the cube or reaches a defined move limit.

The experiment must measure two dimensions:

- **Speed:**
  - Average of how quickly each model returns each move
  - The total time it takes to either solve the solve the Rubik's Cube or execute the maximum quoted moves.
- **Thinking quality:**
  - Whether or not the model solves the Rubik's Cube under maximum quoted moves.
  - How many total moves it uses compared with the known shortest solution.

Maximum moves a model can make given a state = 30.
(Please note that any scrambled 3x3 Rubik's Cube can be solved in 20 moves or less)

Models to compare

Model	Category	OpenRouter model ID
Jev 1.13	System-1 decision model	typesafe/jev-1.13
OpenAI GPT-6 Luna Decisions	System-1 Decision model	openai/gpt-6-luna-decisions
Cloudflare Clef	System-1 Decision model	cloudflare/clef
Claude Opus 5.5	Frontier LLM	anthropic/claude-opus-5.5

Model names, availability, routing, API support, and identifiers can change. Verify the current model pages in OpenRouter before running the benchmark. The first three models are evaluated using the Decisions API interface; Opus is evaluated through chat completions, with both interfaces adapted to the same game-action contract.

2. Learning objectives

1. Integrate decision models and frontier LLMs into one application through OpenRouter.
2. Measure response latency and end-to-end agent execution time.
3. Evaluate decision quality using solve rate, move count, and shortest-solution benchmarks.
4. Design a fair experiment where all models receive the same initial problem state and rules.
5. Analyze the trade-off between fast decisions and high-quality solutions.

3. Application to build

A. Game interface

Display a scrambled 3×3 Rubik's Cube, a button to run the benchmark, the selected model's current move, a move history, and the final result. Clearly indicate whether the cube is solved or the model reached its move limit.

B. Deterministic cube engine

Generate a valid scramble, serialize the cube state, apply each model-selected move, and verify whether all six faces are solved. Every model must use the same engine and move definitions.

C. Statistical Benchmark dashboard

for each model show

- Per-move average latency
- Total solve time in seconds or minutes
- total number of moves
- success rate (percentage, how much Rubik's Cube is solved)
- Optimal GAP defined in section 7

Suggested implementation stack

Python + FastAPI backend, a simple React or HTML/JavaScript frontend, and an OpenRouter integration layer. Participants may use an established cube library rather than implementing cube rotations from scratch.

4. How the AI plays the game

Remember the example of Snake game, we have in classroom. The entire gaming environment must be transformed into a state/action space. Model will receive a state and will be asked to perform an action to get closer to the solution state.

5. Integrating the models through OpenRouter

Decision models: Jev, GPT-6 Luna Decisions, and Clef

Call the OpenRouter Decisions API with the current state and a typed question whose options represent the allowed moves. Extract the selected action from the structured response and pass it to the game engine. Record any confidence or probability field if the response supplies one.

Endpoint:

```
POST https://openrouter.ai/api/alpha/decisions
```

Use the same question wording and option definitions across the three decision models, changing only the model identifier. Confirm the current Decisions API schema and response structure in OpenRouter documentation before implementation. Abbreviated request example

```
{
  "model": "typesafe/jev-1.13",
  "state": {
    "cube_facelets": "<current 54-character state>",
    "recent_moves": ["R", "U'", "F2"]
  },
  "questions": {
    "next_move": {
      "type": "choice",
      "instructions": "Choose the single next move most likely to progress toward the solved cube.",
      "criteria": {
        "U": "Turn the upper face clockwise by 90 degrees",
        "U'": "Turn the upper face counterclockwise by 90 degrees",
        "U2": "Turn the upper face by 180 degrees"
      }
    }
  }
}
```

Frontier LLM: Claude Opus 5.5

Use OpenRouter's standard chat-completions API. Send the current cube state, the permitted actions, and the same recent-move history policy. Require exactly one valid move in a constrained JSON response.

Endpoint:

```
POST https://openrouter.ai/api/v1/chat/completions
```

Model identifier:

```
anthropic/claude-opus-5.5
```

Example expected response format:

```
{
  "move": "R'"
}
```

Validate the returned move before applying it. Log and count malformed responses instead of silently converting them into actions. Use structured output constraints if available for the selected model and route.

API-key security

Keep the OpenRouter API key in a backend environment variable. Never put the key in frontend code or expose it to the browser. Do not commit real keys to the repository.

6. Experiment design

Repeated puzzles, consistent rules, and a shortest-path reference are necessary for a meaningful comparison.

Test set	Scramble length	Purpose
Warm-up	3–4 random turns	Verify model adapters, move validation, and the cube engine.
Primary benchmark	4–5 random turns	Compare models on small puzzles where exact shortest distances can be computed.
Extension	15–20 random turns	Probe behavior on more complex states and observe failure rates.

Minimum recommended sample: five primary puzzles, with all four models run on each puzzle, giving 20 model-puzzle trials. For stronger evidence, use 10 or more puzzles and repeat experiments.

- Set the maximum move limit, such as 30 moves.
- Reaching the limit is an unsuccessful attempt, not a solved attempt.
- Use the same timeout and retry policy for every model. Log retries and invalid outputs explicitly.

7. Required measurements

Establishing the true shortest solution

For the primary benchmark, use breadth-first search or another complete shortest-path method on the small-state test suite to establish the exact distance to the solved cube under the chosen move metric. Then calculate:

Optimality gap = Model moves − Exact shortest moves

An optimality gap of zero means the model solved the puzzle in the minimum number of moves under the chosen metric.

8. Submission requirements

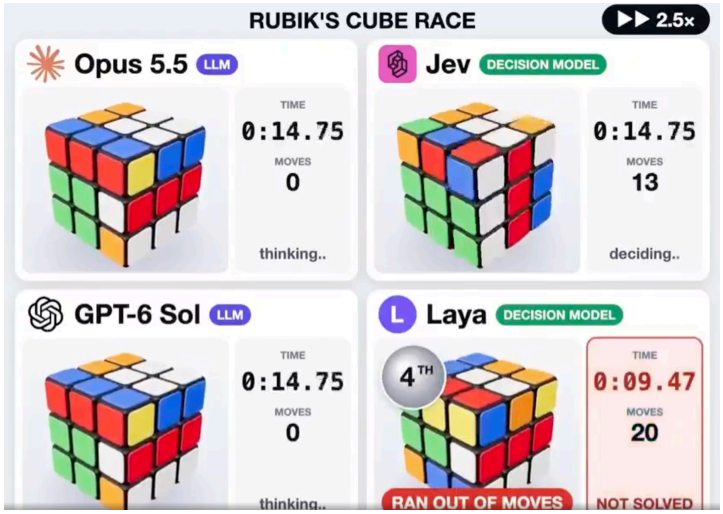
1. Working application

Submit source code, setup instructions, an environment-variable template, as GitHub repository.

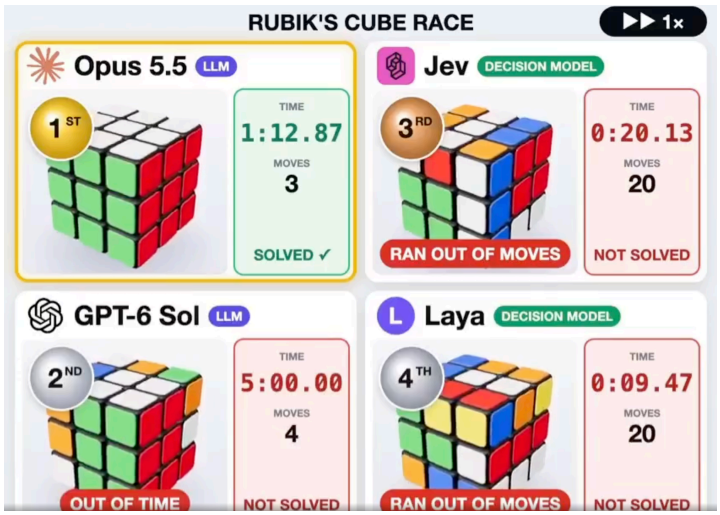
2. Live demonstration: Screen Recording

As a demo, submit a screen-recording of your application, demonstrating the models playing under the Rubik's Cube game.
Submitted screen recording must address the **4 points listed below**:
(Submissions that do not comply with these 4 instructions will be marked invalid.)

1. In-Progress Stage: Models are playing the Game



2. Completion Stage: Models have completed the Game.



3. In the end of the screen recording there must be a slide, demonstrating the statistical benchmarks defined in section 3.
4. **Student name and roll number must be clearly visible on-screen during the entire demo.** This instruction is important: Failure to include this detail will render the submission invalid.

Questions the experiment must answer clearly answer

1. *Which model is fastest in-terms of making decisions?*
2. *Which model achieved the highest solve rate?*
3. *Which model found the shortest solutions, and how close were they to the known optimum?*
4. *Did the fastest model also achieve the best solution quality?*
5. *Would you use the fastest decision model, the frontier LLM, or a hybrid agent in a real-world application? What evidence supports your choice?*